ReviveOS

SOFTWARE RUNTIME RESURRECTION

使用方案

从提交仓库到下载胶囊

Web 控制台 · CLI · API / CI · 生产部署

2026.10 · 生产部署版

目录

1. 先选入口
2. 使用前准备
3. Web 控制台完整流程
4. CLI 使用
5. API 与 CI
6. 生产部署
7. 任务内部限制
8. 典型使用方案
9. 排障
10. 验收清单
11. 代码依据

> **最短路径**
> Web：登录 → 提交仓库 / 预算 / 超时 → 看事件 → 打开项目 → 下载胶囊
> CLI：analyze → resurrect → verify / replay
> API：创建项目 → resurrect → SSE → 下载胶囊

这份方案按“拿到一个仓库之后怎么用”来写。先选入口，再按步骤提交、观察、检查、下载；最后给出团队和 CI 的落地方式。

1. 先选入口

场景	入口	适合谁	产出
第一次体验	Web 控制台 `/login`	不写代码的使用者	任务、报告、胶囊
本机排查	CLI `revive analyze` / `resurrect --local`	开发者	环境基因、`.revive` 胶囊
团队任务	Web 控制台 + 共享控制面	团队管理员、维护者	可交接的任务与胶囊
CI / 批处理	REST API + API Key	自动化脚本	任务状态、胶囊文件
桌面使用	Electron 桌面端	Windows 用户	同 Web 控制台

Web 和桌面端不需要在本机安装 Docker；Docker 只用于执行侧。CLI 的 `--local` 模式需要 Docker，或在明确信任代码的前提下显式允许宿主执行。

2. 使用前准备

2.1 账号与权限

- 通过 `/login` 注册或登录。密码至少 8 位。
- 普通用户可以创建项目、提交任务、查看自己的任务和胶囊。
- 模型配置、维护开关、全局任务管理只有管理员可见。
- 第一个管理员需要先注册普通账号，再由部署者在数据库中提升角色：

```
UPDATE users SET role = 'admin' WHERE email = 'admin@example.com';
```

2.2 仓库要求

- Web 提交只接受公开 GitHub 仓库，格式为 `https://github.com/owner/repo`。
- 默认仓库大小上限 200 MB，可在部署时调整。
- 代码必须是你拥有分析、构建和运行授权的项目。
- 仓库里最好有依赖清单、构建脚本或测试；没有测试也能跑，但行为比较会缺少基线。

2.3 默认配额

项目	默认值	调整变量
并发任务	2	`REVIVE_QUOTA__MAX_CONCURRENT_JOBS`
每日任务	20	`REVIVE_QUOTA__MAX_JOBS_PER_DAY`
仓库大小	200 MB	`REVIVE_QUOTA__MAX_REPO_MB`
单任务预算	页面默认 5 USD，接口上限 1000	提交时填写
单任务超时	页面默认 60 分钟，接口上限 86400 秒	提交时填写
胶囊大小	64 MB	`REVIVE_HTTP__MAX_CAPSULE_BYTES`
JSON 请求体	2 MB	`REVIVE_HTTP__MAX_REQUEST_BYTES`
登录失败锁定	5 次 / 15 分钟	`REVIVE_RATE__LOGIN_FAILURE_LIMIT`
Access Token	15 分钟	`REVIVE_JWT__ACCESS_MINUTES`
Refresh Token	7 天	`REVIVE_JWT__REFRESH_DAYS`
API Key	90 天	`REVIVE_API_KEY__TTL_DAYS`

3. Web 控制台完整流程

3.1 登录

1. 打开 `https://<你的域名>/login`，本地开发通常是 `http://127.0.0.1:3000/login`。
2. 没有账号时切换到“注册”，填写邮箱和密码。
3. 登录成功后进入 `/` 总览页。右上角显示“控制面在线”才表示 API 可达。
4. 如果控制面离线，先检查设置页里的 API 地址，或确认 `/healthz` 可访问。

访问令牌和刷新令牌只放在当前标签页的 `sessionStorage`，关闭标签页即失效；控制台不会把令牌写进 URL。

3.2 提交一次复生任务

在总览页的“提交复生任务”表单里：

1. **GitHub 仓库地址**：填写 `https://github.com/owner/repo`。
2. **预算上限（USD）**：默认 `5`，按任务填写。
3. **超时（分钟）**：默认 `60`。
4. 点击“提交复生”。

提交后系统会做两件事：先创建项目，再创建任务。页面跳转到 `/jobs/<job_id>`。

常见返回：

情况	页面表现	处理
仓库地址格式不对	输入框下方提示	改成 `https://github.com/owner/repo`
项目已存在（409）	提示“项目已存在”	点“为该项目新建任务”，或“查看项目历史”
并发或每日配额用尽（429）	显示配额提示和倒计时	等 `Retry-After` 或次日再提交
仓库超过大小上限（413）	提示仓库过大	换更小的仓库，或由管理员调高上限
控制面维护中（503）	提示暂停接单	等维护结束

3.3 观察任务

任务页 `/jobs/<job_id>` 会自动订阅 SSE：

```
GET /api/v1/jobs/{job_id}/events/stream
```

页面会显示：

- 当前任务状态：`queued`、`running`、`succeeded`、`failed`、`cancelled`。
- 已接收事件数、当前事件类型、创建时间、结束时间。
- 事件时间线；SSE 断线时自动重连，超过重试次数会降级为每 5 秒轮询。
- “刷新任务资源”“取消任务”“打开项目”“立即重连”。

常见事件：

事件	含义
`job.queued`	任务已入队
`archaeology.started`	开始环境考古
`env.inferred`	环境基因已推断
`sandbox.created`	沙箱工作区已创建
`behavior.compared`	行为比较完成或记录缺失
`score.computed`	评分完成
`capsule.created`	胶囊已生成
`job.succeeded`	任务终态：成功
`job.failed`	任务终态：失败

任务成功后会显示“查看复生评分”和“打开胶囊”两个入口。

3.4 查看项目材料

任务页点“打开项目”进入 `/projects/<project_id>`。项目页按标签拆开：

标签	路径	用途
总览	`/projects/{id}`	仓库、状态、最近任务、评分概览
环境基因	`/projects/{id}/genome`	Python、OS、依赖、置信度
证据	`/projects/{id}/evidence`	依赖、CI、Dockerfile 等来源
构建	`/projects/{id}/builds`	构建尝试与失败位置
补丁	`/projects/{id}/patches`	修复 diff、文件范围、行数
行为	`/projects/{id}/behavior`	基线比较与行为指纹
报告	`/projects/{id}/report`	评分、报告与结论
胶囊	`/projects/{id}/capsules`	该项目生成的胶囊列表

建议顺序：先看环境基因和证据，确认环境是否可信；再看构建和补丁，确认修改范围；最后看行为与评分，确认结果依据。

3.5 下载胶囊

在 `/capsules/<capsule_id>` 或项目胶囊列表里：

1. 查看 manifest：`format`、`source.repo`、`source.commit`、`genome_ref`、`behavior.compatibility`、`sbom_digest`。
2. 点“下载 .revive”。
3. 浏览器通过 API 的 presigned 302 下载，文件名为 `<capsule_id>.revive`。

胶囊通常包含 manifest、Genome、依赖记录、Dockerfile、补丁、行为计划与指纹、SBOM、provenance、source.tar 等条目。具体条目以 manifest 为准。

3.6 设置页

`/settings` 包括：

- **控制面地址**：浏览器版保存到 localStorage，桌面版交给 Electron 主进程持久化。
- **我的资料与配额**：邮箱、角色、并发上限、每日上限。
- **API Key**：新建时明文只显示一次；列表只显示前缀；可以删除。
- **外观**：壁纸只保存在本机，不上传服务端。
- **模型提供方**：只有管理员可改；保存后由 worker 在下一个任务读取。
- **安全说明**：令牌、密钥、壁纸的存储边界。

模型提供方支持：

提供方	用途	必填
`fake`	本地演示，不联网	无
`openai`	OpenAI 兼容接口	模型名、API Key
`anthropic`	Anthropic 接口	模型名、API Key
`custom`	OpenAI 兼容网关	Base URL、模型名、API Key

“推理强度”可选 `low` / `medium` / `high`；“测试连接”会发起一次真实探测。API Key 只回显末四位，完整值不再返回。

---

4. CLI 使用

4.1 入口

```
# 在仓库根目录
uv run revive --help
# Windows 虚拟环境
.venv/Scripts/revive.exe --help
```

CLI 命令：

命令	用途
`revive analyze`	只做环境考古，输出环境基因
`revive resurrect`	本地复生，或提交到控制面
`revive verify`	检查胶囊结构和摘要
`revive replay`	进入胶囊工作区执行命令
`revive login`	保存控制面凭据
`revive jobs ls` / `jobs get`	查看服务端任务
`revive download`	下载服务端胶囊

4.2 只做环境考古

```
uv run revive analyze https://github.com/owner/repo --out genome.json
# 不查询 PyPI 历史版本
uv run revive analyze ./local-repo --offline
```

输出包括 Python 版本、OS、依赖、证据来源和 `overall_confidence`。置信度低于 `0.4` 时命令以退出码 3 结束，表示证据不足。

4.3 本地复生

```
uv run revive resurrect ./local-repo \
  --local \
  --budget-usd 5 \
  --timeout 600 \
  --out demo.revive
```

可用参数：

参数	说明
`--local`	使用本地流水线
`--budget-usd`	LLM 预算上限
`--timeout`	流水线超时秒数
`--out`	输出 `.revive` 路径
`--work-root`	沙箱工作目录
`--keep-sandbox`	保留沙箱工作区，便于排查
`--allow-host-execution`	Docker 不可用时允许在宿主执行目标代码

`--allow-host-execution` 只应对你信任的代码使用。生产环境应使用 Docker 执行端口，不要用宿主执行替代隔离。

4.4 提交到控制面

先保存凭据：

```
uv run revive login \
  --server https://revive.example.com \
  --token "$REVIVE_ACCESS_TOKEN"
# 或用邮箱密码
uv run revive login --server https://revive.example.com --email you@example.com --password '...'
```

提交并实时看事件：

```
uv run revive resurrect https://github.com/owner/repo \
  --server https://revive.example.com \
  --budget-usd 5 \
  --timeout 3600
```

任务结束后：

```
uv run revive jobs ls
uv run revive jobs get <job_id>
uv run revive download <job_id> -o capsule.revive
```

`REVIVE_SERVER_URL` 可以替代重复的 `--server`。凭据保存在系统钥匙串，或权限为 0600 的回退文件。

4.5 检查与重放胶囊

```
uv run revive verify capsule.revive
uv run revive verify capsule.revive --json
uv run revive replay capsule.revive -- python -m pytest -q
```

- `verify` 输出 manifest、摘要和可用的重建检查报告，并按报告中的 `exit_code` 退出。没有可用重建器时只完成结构与摘要检查。
- `replay` 展开胶囊工作区并执行你给出的命令；需要 Docker，或显式加 `--allow-host-execution` 在宿主执行。
- 这两个命令都不会修改原胶囊。

4.6 CLI 退出码

退出码	含义
`0`	成功
`1`	任务失败，或本地复生状态为 `dead`
`2`	参数、凭据、网络、Docker 或执行错误
`3`	环境证据不足（`analyze` 置信度低于 0.4）

`verify` 的退出码来自报告，可能是 `0` 或 `2`；以报告为准。

---

5. API 与 CI

5.1 认证

两种方式，任选其一：

```
Authorization: Bearer <access_token>
```

或：

```
X-Revive-Key: rev_api_key_...
```

API Key 在设置页创建，明文只返回一次。不要把它写进仓库、URL 或日志。

5.2 创建项目并提交任务

```
BASE=https://revive.example.com
TOKEN=...                    # 登录后的 access_token
REPO=https://github.com/owner/repo

PROJECT=$(curl -fsS -X POST "$BASE/api/v1/projects" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d "{\"repo_url\":\"$REPO\"}")
PID=$(printf '%s' "$PROJECT" | python -c 'import json,sys; print(json.load(sys.stdin)["id"])')

JOB=$(curl -fsS -X POST "$BASE/api/v1/projects/$PID/resurrect" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"budget_usd":5,"timeout_seconds":3600}')
JID=$(printf '%s' "$JOB" | python -c 'import json,sys; print(json.load(sys.stdin)["job_id"])')
```

如果项目已存在，`POST /projects` 返回 409，并带 `existing_id`；直接用这个 id 提交新任务。

5.3 订阅事件

```
curl -N -H "Authorization: Bearer $TOKEN" \
  "$BASE/api/v1/jobs/$JID/events/stream"
```

SSE 支持 `Last-Event-ID`，断线重连时可从上次事件继续。没有 SSE 客户端时，也可以轮询：

```
curl -fsS -H "Authorization: Bearer $TOKEN" "$BASE/api/v1/jobs/$JID"
```

5.4 取胶囊并下载

```
CAPSULES=$(curl -fsS -H "Authorization: Bearer $TOKEN" \
  "$BASE/api/v1/projects/$PID/capsules?limit=1")
CID=$(printf '%s' "$CAPSULES" | python -c 'import json,sys; print(json.load(sys.stdin)["items"][0]["id"])')

# -L 跟随 presigned 302
curl -fL -H "Authorization: Bearer $TOKEN" \
  -o capsule.revive "$BASE/api/v1/capsules/$CID/download"
```

取消任务：

```
curl -fsS -X POST -H "Authorization: Bearer $TOKEN" \
  "$BASE/api/v1/jobs/$JID/cancel"
```

5.5 CI 示例

```
#!/usr/bin/env bash
set -euo pipefail

BASE="${REVIVE_SERVER_URL:?}"
KEY="${REVIVE_API_KEY:?}"
REPO="${1:?usage: ci-resurrect.sh <github-repo-url>}"

PROJECT=$(curl -fsS -X POST "$BASE/api/v1/projects" \
  -H "X-Revive-Key: $KEY" -H 'Content-Type: application/json' \
  -d "{\"repo_url\":\"$REPO\"}")
PID=$(printf '%s' "$PROJECT" | python -c 'import json,sys; print(json.load(sys.stdin)["id"])')

JOB=$(curl -fsS -X POST "$BASE/api/v1/projects/$PID/resurrect" \
  -H "X-Revive-Key: $KEY" -H 'Content-Type: application/json' \
  -d '{"budget_usd":5,"timeout_seconds":3600}')
JID=$(printf '%s' "$JOB" | python -c 'import json,sys; print(json.load(sys.stdin)["job_id"])')

for _ in $(seq 1 720); do
  STATE=$(curl -fsS -H "X-Revive-Key: $KEY" "$BASE/api/v1/jobs/$JID")
  STATUS=$(printf '%s' "$STATE" | python -c 'import json,sys; print(json.load(sys.stdin)["status"])')
  case "$STATUS" in
    succeeded) break ;;
    failed|cancelled) echo "job $STATUS" >&2; exit 1 ;;
  esac
  sleep 5
done

CAPSULES=$(curl -fsS -H "X-Revive-Key: $KEY" \
  "$BASE/api/v1/projects/$PID/capsules?limit=1")
CID=$(printf '%s' "$CAPSULES" | python -c 'import json,sys; print(json.load(sys.stdin)["items"][0]["id"])')
curl -fL -H "X-Revive-Key: $KEY" -o capsule.revive \
  "$BASE/api/v1/capsules/$CID/download"
```

5.6 常见 HTTP 状态

状态	含义	处理
`400`	请求字段无效	检查 JSON 和字段范围
`401`	未认证或令牌过期	重新登录，或刷新令牌
`403`	权限不足（如非管理员）	换管理员账号
`404`	项目、任务或胶囊不存在	检查 id
`409`	项目已存在	使用响应里的 `existing_id`

状态	含义	处理
`413`	仓库或请求体过大	换仓库或调高上限
`429`	并发、每日或登录频率超限	按 `Retry-After` 等待
`503`	维护中	等维护结束

6. 生产部署

6.1 拓扑

```
Caddy (443 / TLS)
  ├─ /api/*      → FastAPI
  ├─ /healthz    → FastAPI
  ├─ /readyz     → FastAPI
  └─ 其他         → Web 静态控制台

FastAPI → PostgreSQL / Redis / S3
Celery worker → Docker sandbox (唯一执行用户代码的进程)
Celery beat → 清理孤儿沙箱
```

API 不执行用户代码。生产环境的 worker 挂载 Docker socket，等同于宿主 root 权限，必须部署在专用、隔离的主机上。

6.2 首次部署

```
# 1. 生产应用配置
cp infra/envs/revive.prod.env.template infra/envs/revive.prod.env
# 用 secret manager 填入 JWT、worker token、数据库/Redis 口令、S3 凭据

# 2. Compose 配置
cp infra/envs/compose.env.template .env
# 填入镜像名、digest、域名、Docker GID、数据库/Redis 口令

# 3. 静态校验、拉取、启动
docker compose --env-file .env -f infra/compose/prod.docker-compose.yml config --quiet
docker compose --env-file .env -f infra/compose/prod.docker-compose.yml pull
docker compose --env-file .env -f infra/compose/prod.docker-compose.yml up -d
docker compose --env-file .env -f infra/compose/prod.docker-compose.yml ps
```

`REVIVE_ENV=prod` 是 fail-closed 的：缺少 `DATABASE_URL`、`REDIS_URL`、`S3_*` 或持久化适配器时 API 拒绝启动。

6.3 首个管理员

1. 打开 `/login` 注册普通账号。
2. 在 PostgreSQL 中提升角色：

```
docker compose --env-file .env -f infra/compose/prod.docker-compose.yml \
  exec postgres psql -U revive -d revive \
  -c "UPDATE users SET role='admin' WHERE email='admin@example.com';"
```

6.4 健康与就绪

```
curl -fsS https://<你的域名>/healthz
curl -fsS https://<你的域名>/readyz
```

- `/healthz` 只表示 API 进程存活，容器健康检查用它。
- `/readyz` 检查 PostgreSQL、Redis 和对象存储；负载均衡或编排系统用它决定是否接流量。
- `/metrics` 是 Prometheus 文本，只在内网暴露，Caddy 不代理。

6.5 备份与恢复

```
BACKUP_DIR=/backups \
DATABASE_URL=postgresql://... \
bash infra/backup/pg-backup.sh

BACKUP_FILE=/backups/revive-*.sql.gz \
DATABASE_URL=postgresql://... \
bash infra/backup/pg-restore.sh
```

备份必须复制到异地或独立账号/卷，不能与生产同机同卷。S3 开启版本控制，并定期做恢复演练。恢复后抽查项目、任务、胶囊和 `revive verify`。

6.6 升级与回滚

1. 打 tag，发布 API、worker、Web 三个镜像到 GHCR；Trivy 扫描和 CycloneDX SBOM 作为发布门禁。
2. 把新镜像和依赖镜像固定到 digest，更新 `.env` 中的 `*_DIGEST`。
3. `docker compose pull`、`up -d`；migrate 服务会先执行 Alembic 到 head。
4. 保留上一组 digest；回滚时切回旧 digest，数据库按备份/迁移策略恢复。
5. 升级后检查 `/readyz`、worker 日志、beat 清理任务、胶囊下载和一次完整任务。

6.7 安全边界

- Caddy 是唯一入口，只暴露 80/443；API 端口不要直接发布。
- worker 主机专用，限制 operator 权限；Docker socket 不是多租户隔离边界。

- `REVIVE_SECRETS__KEY` 在 API 和 worker 之间保持一致；轮换会导致已保存的模型密钥不可读。
- API Key 只显示一次，按用户撤销；令牌不要写进 URL、日志或仓库。
- 模型密钥只写入服务端运行时配置，接口只回显末四位。

---

7. 任务内部限制

这些限制决定了一次任务最多能修到什么程度，提交前应知道：

限制	值
单次补丁文件数	最多 10 个文件
单次补丁行数	最多 300 行增删
构建尝试	最多 5 次
修复调用	最多 4 次
测试文件	禁止修改
构建失败	恢复上一版快照
沙箱	非 root、只读根文件系统、运行时禁网、资源限制

补丁范围按实际 diff 计数，不是按模型自述。测试失败会终止自动流程，不会无限修复。

---

8. 典型使用方案

8.1 个人第一次使用

1. 打开 Web，注册登录。
2. 提交一个 200 MB 以内的公开 Python 仓库。
3. 预算 5 USD、超时 60 分钟。
4. 在任务页看 `env.inferred`、`sandbox.created`、`capsule.created`。
5. 打开项目页，先看环境基因和证据，再看构建、补丁、行为、报告。
6. 下载 `.revive`，用 `revive verify` 检查结构。

8.2 团队接入

1. 管理员在设置页配置模型提供方并点“测试连接”。

2. 为 CI 单独创建 API Key，设置备注，按需撤销。
3. 根据团队规模设置并发和每日配额；生产 worker 放独立主机。
4. 约定胶囊命名和归档位置；发布前跑一次备份恢复演练。
5. 用 `/readyz` 接负载均衡，用 `/metrics` 接监控。

8.3 CI 自动化

1. 在 CI secret 中保存 `REVIVE_SERVER_URL` 和 `REVIVE_API_KEY`。
2. 按第 5.5 节脚本创建项目、提交任务、轮询终态。
3. 成功时下载 `capsule.revive` 作为构建产物；失败时保留任务 id 和事件日志。
4. 不要把 API Key 打印到日志；不要把 `.revive` 当唯一备份，数据库和 S3 仍要备份。

8.4 研究与归档

1. `revive analyze ./repo --offline --out genome.json` 先固定环境基因。
2. `revive resurrect ./repo --local --out archive.revive` 生成胶囊。
3. `revive verify archive.revive --json` 保存校验报告。
4. `revive replay archive.revive -- <命令>` 在胶囊工作区复跑关键命令。
5. 把 genome、胶囊、校验报告和命令记录一起归档。

---

9. 排障

现象	优先检查	处理
控制面离线	API 地址、`/healthz`	改设置页地址，或启动 API
登录后立刻 401	标签页是否关闭、令牌是否过期	重新登录；不要跨标签复制 sessionStorage
403	当前账号角色	模型/维护接口需要管理员
409	项目是否已存在	用 `existing_id` 新建任务
413	仓库大小	换小仓库或调高 `MAX_REPO_MB`
429	并发/每日配额、登录失败	按 `Retry-After` 等待；管理员调配额
503	维护开关	等维护结束
任务失败	任务事件、构建日志、补丁	看失败阶段；不要只看“成功/失败”
SSE 断开	网络/代理	页面会重连；降级后每 5 秒轮询
下载 404	胶囊是否生成	任务未成功或胶囊未上传时没有下载

现象	优先检查	处理
本地 `resurrect` 报 Docker 不可用	Docker daemon	启动 Docker；或只在可信代码上加 `--allow-host-execution`
模型测试失败	Base URL、模型名、Key	`custom` 必须填 Base URL；检查网关兼容性
胶囊 verify 非 0	报告中的检查项	看 manifest/digests/rebuild 哪一项失败

10. 验收清单

- ☐ `/healthz` 返回 200。
- ☐ `/readyz` 返回 200，且 PostgreSQL、Redis、对象存储均为就绪。
- ☐ 能注册、登录、退出；刷新令牌可用。
- ☐ 提交公开 GitHub 仓库后任务进入 `queued` / `running`。
- ☐ 任务页能收到 SSE 事件；断线后能重连或降级轮询。
- ☐ 项目页能看到环境基因、证据、构建、补丁、行为、报告。
- ☐ 任务成功后能打开胶囊并下载 `.revive`。
- ☐ `revive verify` 能输出报告；`revive replay` 能执行指定命令。
- ☐ 管理员能保存并测试模型配置；非管理员看不到模型配置。
- ☐ 配额、维护开关、API Key 撤销生效。
- ☐ 备份脚本能备份并恢复到临时库。
- ☐ 生产 Compose 能通过 `config --quiet`，Caddy 只暴露 80/443。

11. 代码依据

功能	位置
Web 提交表单	`apps/web/src/features/submit/ResurrectForm.tsx`
Web 任务页与 SSE	`apps/web/app/jobs/[id]/ClientPage.tsx`
Web 胶囊下载	`apps/web/app/capsules/[id]/ClientPage.tsx`
Web 设置与模型配置	`apps/web/app/settings/page.tsx`、`apps/web/src/features/settings/ModelProviderSection.tsx`
API 路由与任务提交	`apps/api/src/revive_api/main.py`
配额与配置默认值	`apps/api/src/revive_api/config.py`

功能	位置
任务执行	`apps/api/src/revive_api/tasks/pipeline.py`
CLI 命令	`cli/src/revive_cli/main.py`
CLI 服务端客户端	`cli/src/revive_cli/client.py`
生产编排	`infra/compose/prod.docker-compose.yml`
Caddy 路由	`infra/compose/caddy/Caddyfile`
备份脚本	`infra/backup/pg-backup.sh`、`infra/backup/pg-restore.sh`
部署与就绪说明	`docs/production-readiness.md`、`docs/runbook-deploy.md`